

UNIDAD 3

Algoritmos de Búsqueda y Ordenamiento

Conceptos básicos y ejemplos en Python

Facultad de Ingeniería

Agenda de la sesión

Distribución sugerida — 120 minutos

10 min

Introducción y motivación

¿Por qué importan estos algoritmos?

25 min

Algoritmos de búsqueda

Lineal y binaria — análisis y código

10 min

Receso / actividad corta

Ejercicio en parejas

50 min

Algoritmos de ordenamiento

6 algoritmos con demostración paso a paso

15 min

Aplicaciones prácticas con datos

Python nativo, NumPy y pandas

10 min

Cierre, dudas y tarea

Ejercicios propuestos

Objetivos de aprendizaje

Al concluir la sesión, el estudiante será capaz de:

1

Comprender

los algoritmos de búsqueda lineal y binaria y sus condiciones de uso.

2

Implementar

en Python los algoritmos de ordenamiento: selección, inserción, burbuja, quicksort y mergesort.

3

Analizar

la complejidad temporal y espacial usando la notación O grande.

4

Aplicar

funciones de búsqueda y ordenamiento usando Python nativo, NumPy y pandas.

¿Por qué importan estos algoritmos?

Casi todo software realiza, en algún momento, dos operaciones fundamentales:



BUSCAR

Localizar información dentro de una colección



ORDENAR

Reorganizar datos según un criterio útil

Ejemplos del día a día

- Buscar un contacto en el celular
- Filtrar correos por remitente
- Ordenar archivos por fecha
- Listar productos por precio
- Encontrar la calificación más alta
- Indexar millones de páginas (Google)
- Ordenar resultados de una consulta SQL

Notación O grande (Big-O)

Cómo medir la eficiencia de un algoritmo

La notación O describe cómo crece el tiempo de ejecución en función del tamaño de la entrada (n). Permite comparar algoritmos sin depender del hardware o el lenguaje.

$O(1)$	Constante	<i>Acceder a un elemento por índice</i>
$O(\log n)$	Logarítmica	<i>Búsqueda binaria</i>
$O(n)$	Lineal	<i>Búsqueda lineal</i>
$O(n \log n)$	Linealítmica	<i>Quicksort, Mergesort</i>
$O(n^2)$	Cuadrática	<i>Burbuja, Selección, Inserción</i>

💡 A mayor crecimiento, peor desempeño conforme aumenta n .

PARTE 3.1

Algoritmos de búsqueda

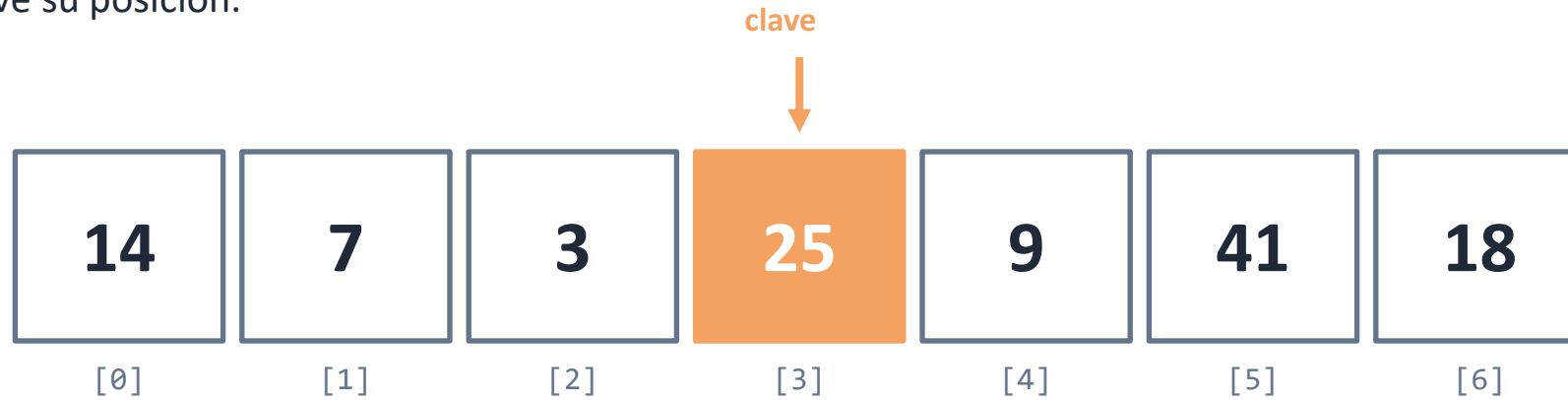
Lineal y binaria

🕒 Tiempo estimado: 25 minutos

¿Qué es un algoritmo de búsqueda?

Procedimiento que, dada una colección de elementos y un valor objetivo (la clave), determina si la clave está presente y, en caso afirmativo, devuelve su posición.

Lista de entrada:



Dos estrategias fundamentales:

Búsqueda lineal

Recorre uno por uno • Sin orden previo

Búsqueda binaria

Divide a la mitad • Requiere lista ordenada

Búsqueda lineal (secuencial)

Recorre la colección elemento por elemento

¿Cómo funciona?

1. Empieza en la posición 0
2. Compara con la clave buscada
3. Si coinciden → devuelve el índice
4. Si no, avanza una posición
5. Repite hasta encontrarla o llegar al final
6. Si no la encuentra, devuelve -1

Características

- ✓ No requiere datos ordenados
- ✓ Funciona con cualquier iterable
- ✗ Lento en colecciones grandes



```
def busqueda_lineal(lista, clave):  
    for indice in range(len(lista)):  
        if lista[indice] == clave:  
            return indice  
    return -1  
  
# Ejemplo  
numeros = [14, 7, 3, 25, 9, 41, 18]  
print(busqueda_lineal(numeros, 25))  
# Salida: 3
```

Búsqueda lineal — traza paso a paso

Buscando el 25 en [14, 7, 3, 25, 9, 41, 18]

Paso 1: lista[0] = 14 ≠ 25 → continuar

14	7	3	25	9	41	18
----	---	---	----	---	----	----

Paso 2: lista[1] = 7 ≠ 25 → continuar

14	7	3	25	9	41	18
----	---	---	----	---	----	----

Paso 3: lista[2] = 3 ≠ 25 → continuar

14	7	3	25	9	41	18
----	---	---	----	---	----	----

Paso 4: lista[3] = 25 = 25 → ¡encontrado!

14	7	3	25	9	41	18
----	---	---	----	---	----	----

Resultado

Encontrado en el índice 3 — 4 comparaciones realizadas

Análisis de la búsqueda lineal

Complejidad temporal y espacial

MEJOR CASO

$O(1)$

La clave está en la primera posición

CASO PROMEDIO

$O(n)$

Se revisa la mitad de la lista en promedio

PEOR CASO

$O(n)$

La clave está al final o no existe



Para una lista de 1,000,000 elementos:

Peor caso lineal: **1,000,000 comparaciones** vs Peor caso binario: **20 comparaciones**

Búsqueda binaria

Divide y vencerás — pero exige lista ordenada

⚠ Requisito fundamental

La búsqueda binaria SOLO funciona en colecciones ordenadas.

Estrategia:

1. Calcular el índice medio del rango actual
2. Comparar lista[medio] con la clave
3. Si son iguales → encontrado
4. Si la clave es menor → buscar en la mitad izquierda
5. Si la clave es mayor → buscar en la mitad derecha
6. Repetir hasta encontrar o que el rango quede vacío

Eficiencia logarítmica

En cada paso reduce a la mitad el espacio de búsqueda.

Para n elementos, requiere a lo más $\lceil \log_2 n \rceil$ comparaciones.

$n = 1,000,000 \rightarrow$ solo 20 pasos

Búsqueda binaria — traza paso a paso

Buscando el 18 en [3, 7, 9, 14, 18, 25, 41]

Paso 1: $\text{izq}=0$, $\text{der}=6$, $\text{medio}=3 \rightarrow 14 < 18 \rightarrow$ buscar a la derecha



Paso 2: $\text{izq}=4$, $\text{der}=6$, $\text{medio}=5 \rightarrow 25 > 18 \rightarrow$ buscar a la izquierda



Paso 3: $\text{izq}=4$, $\text{der}=4$, $\text{medio}=4 \rightarrow 18 = 18 \rightarrow$ ¡encontrado!



Resultado

Encontrado en el índice 4 — solo 3 comparaciones

Búsqueda binaria — implementación

Versión iterativa en Python

```
def busqueda_binaria(lista, clave):
    izquierda = 0
    derecha = len(lista) - 1

    while izquierda <= derecha:
        medio = (izquierda + derecha) // 2

        if lista[medio] == clave:
            return medio
        elif lista[medio] < clave:
            izquierda = medio + 1
        else:
            derecha = medio - 1

    return -1

# Ejemplo
ordenados = [3, 7, 9, 14, 18, 25, 41]
print(busqueda_binaria(ordenados, 18)) # 4
print(busqueda_binaria(ordenados, 10)) # -1
```

Complejidad temporal

Mejor: $O(1)$
Promedio: $O(\log n)$
Peor: $O(\log n)$

Memoria

$O(1)$ iterativa
 $O(\log n)$ recursiva

Cuándo usarla

Cuando se harán muchas búsquedas sobre los mismos datos

Lineal vs Binaria

Comparación rápida

Característica	Búsqueda lineal	Búsqueda binaria
Datos ordenados	No requiere	Obligatorio
Mejor caso	$O(1)$	$O(1)$
Peor caso	$O(n)$	$O(\log n)$
Memoria	$O(1)$	$O(1)$ it. / $O(\log n)$ rec.
Implementación	Muy fácil	Moderada
Cuándo usar	Listas pequeñas o desordenadas	Listas grandes y ya ordenadas

PARTE 3.2

Algoritmos de ordenamiento

Selección, inserción, burbuja, quicksort y mergesort

🕒 Tiempo estimado: 50 minutos

Panorama de algoritmos de ordenamiento

Seis algoritmos clásicos en dos familias

Familia $O(n^2)$

Sencillos, didácticos, lentos en datos grandes

1 Secuencial

2 Selección

3 Inserción

4 Burbuja

Familia $O(n \log n)$

Divide y vencerás, eficientes en datos grandes

5 Quicksort

6 Mergesort

💡 "In-place" = ordena sin copiar la lista • "Estable" = preserva el orden relativo entre iguales

Ordenamiento por selección

En cada pasada busca el mínimo y lo coloca al inicio

Idea:

1. Buscar el mínimo del subarreglo no ordenado
2. Intercambiarlo con el primer elemento del subarreglo
3. Avanzar la frontera del subarreglo ordenado
4. Repetir hasta llegar al penúltimo elemento

Análisis

Mejor / Promedio / Peor: $O(n^2)$
Memoria: $O(1)$ — in-place
Estable: No

Minimiza intercambios (a lo más $n - 1$)

```
def seleccion(lista):  
    n = len(lista)  
    for i in range(n - 1):  
        indice_min = i  
        for j in range(i + 1, n):  
            if lista[j] < lista[indice_min]:  
                indice_min = j  
        if indice_min != i:  
            lista[i], lista[indice_min] = (  
                lista[indice_min], lista[i])  
    return lista  
  
# [64, 25, 12, 22, 11]  
# → [11, 12, 22, 25, 64]
```

Ordenamiento por inserción

Como ordenar cartas en la mano

Idea:

1. Tomar el siguiente elemento como "clave"
2. Compararlo con los elementos ya ordenados
3. Desplazar a la derecha los mayores que la clave
4. Insertar la clave en el hueco resultante
5. Repetir desde la posición 1 hasta el final

Análisis

Mejor: $O(n)$ — lista casi ordenada
Promedio / Peor: $O(n^2)$
Memoria: $O(1)$ — in-place
Estable: Sí ✓

Muy eficiente en arreglos pequeños o casi ordenados

```
def insercion(lista):  
    for i in range(1, len(lista)):  
        clave = lista[i]  
        j = i - 1  
        while j >= 0 and lista[j] > clave:  
            lista[j + 1] = lista[j]  
            j -= 1  
        lista[j + 1] = clave  
    return lista  
  
# [12, 11, 13, 5, 6]  
# → [5, 6, 11, 12, 13]
```

Ordenamiento de burbuja

Los grandes "burbujean" hacia el final

Idea:

1. Comparar pares de elementos adyacentes
2. Si están en orden incorrecto, intercambiarlos
3. Al final de cada pasada, el mayor queda en su lugar
4. Repetir hasta no hacer intercambios

Análisis

Mejor: $O(n)$ — con optimización de bandera
Promedio / Peor: $O(n^2)$
Memoria: $O(1)$ — in-place
Estable: Sí ✓

Ideal para enseñar, no para producción

```
def burbuja(lista):
    n = len(lista)
    for i in range(n - 1):
        hubo_intercambio = False
        for j in range(n - 1 - i):
            if lista[j] > lista[j + 1]:
                lista[j], lista[j + 1] = (
                    lista[j + 1], lista[j])
                hubo_intercambio = True
        if not hubo_intercambio:
            break
    return lista

# [5, 1, 4, 2, 8]
# → [1, 2, 4, 5, 8]
```

Estrategia: divide y vencerás

Base conceptual de quicksort y mergesort

Una técnica algorítmica poderosa que sigue tres pasos:

1

DIVIDIR

Partir el problema en subproblemas más pequeños del mismo tipo.

2

VENCER

Resolver los subproblemas recursivamente (cuando son pequeños, directamente).

3

COMBINAR

Unir las soluciones parciales en una solución del problema original.

Resultado: complejidad $O(n \log n)$

Reducir el problema a la mitad en cada paso produce este nivel de eficiencia.

Quicksort

El más usado en la práctica — Tony Hoare, 1960

Idea:

1. Elegir un elemento como pivote
2. Particionar: menores $\leftarrow$ | pivote | $\rightarrow$ mayores
3. Aplicar quicksort recursivamente a cada partición
4. Concatenar: izquierda + pivote + derecha

Análisis

Mejor / Promedio: $O(n \log n)$ ✓
Peor: $O(n^2)$ $\leftarrow$ pivote mal elegido
Memoria: $O(\log n)$ por la recursión
Estable: No

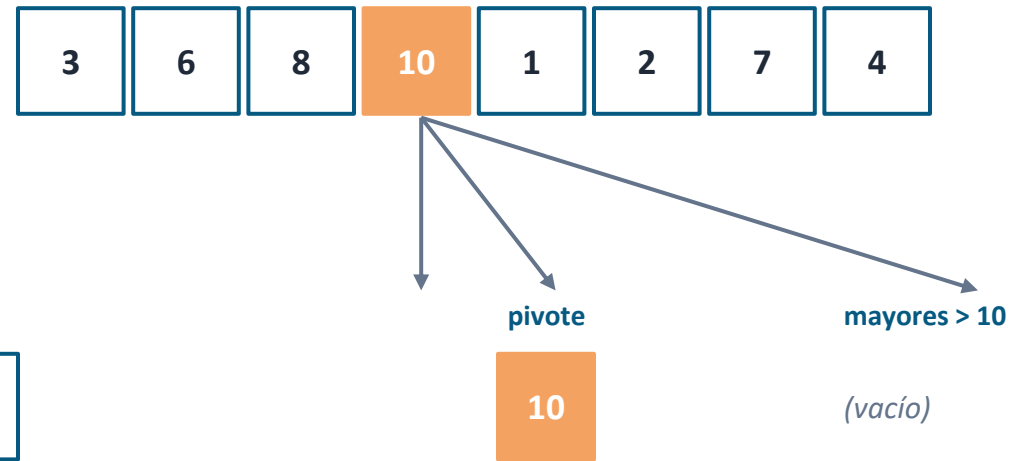
💡 Usar mediana de tres o pivote aleatorio

```
def quicksort(lista):  
    if len(lista) <= 1:  
        return lista  
  
    pivote = lista[len(lista) // 2]  
    menores = [x for x in lista if x < pivote]  
    iguales = [x for x in lista if x == pivote]  
    mayores = [x for x in lista if x > pivote]  
  
    return (quicksort(menores)  
            + iguales  
            + quicksort(mayores))  
  
# [3, 6, 8, 10, 1, 2, 1, 7, 4]  
#  $\rightarrow$  [1, 1, 2, 3, 4, 6, 7, 8, 10]
```

Quicksort — visualización

Partición recursiva sobre [3, 6, 8, 10, 1, 2, 7, 4]

Pivote: 10



menores < 10



Pivote: 8

menores < 8



(continúa la recursión...)

Resultado final: [1, 2, 3, 4, 6, 7, 8, 10]

Mergesort

Divide y fusiona — von Neumann, 1945

Idea:

1. Dividir la lista a la mitad recursivamente
2. Hasta obtener sublistas de un solo elemento
3. Fusionar (merge) pares de sublistas ordenadas
4. Hasta reconstruir la lista completa ordenada

Análisis

Mejor / Promedio / Peor: $O(n \log n)$ ✓
Memoria: $O(n)$ — requiere auxiliares
Estable: Sí ✓

⚡ Predecible y estable, pero usa más memoria

```
def mergesort(lista):
    if len(lista) <= 1:
        return lista

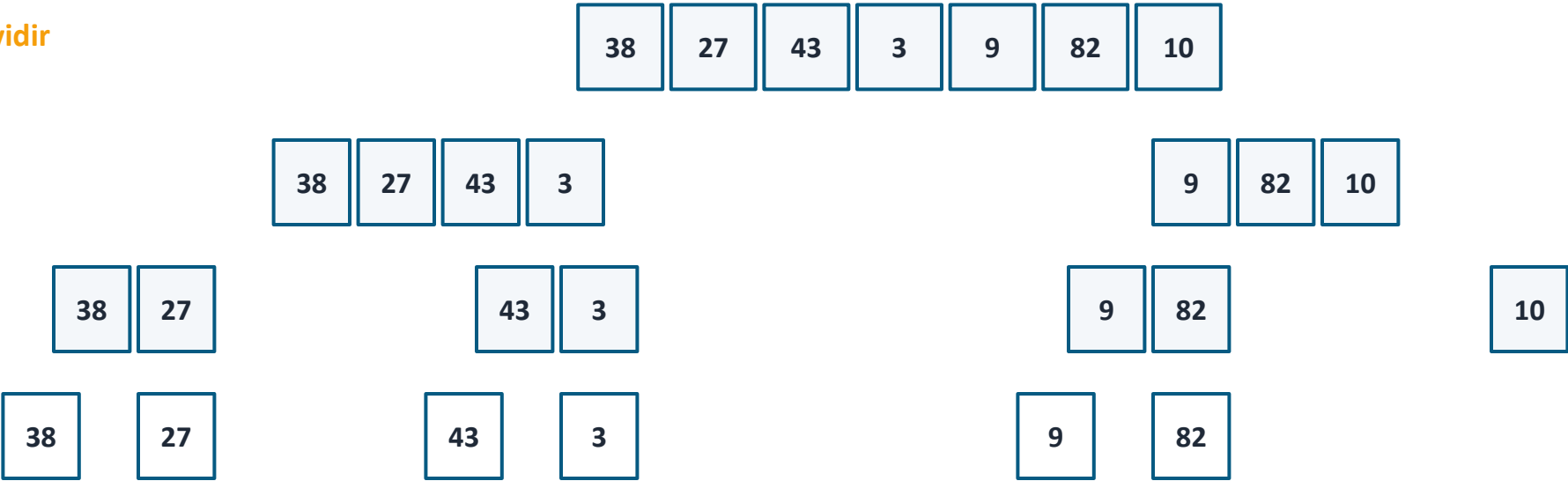
    medio = len(lista) // 2
    izq = mergesort(lista[:medio])
    der = mergesort(lista[medio:])
    return fusionar(izq, der)

def fusionar(izq, der):
    resultado = []
    i = j = 0
    while i < len(izq) and j < len(der):
        if izq[i] <= der[j]:
            resultado.append(izq[i]); i += 1
        else:
            resultado.append(der[j]); j += 1
    resultado.extend(izq[i:])
    resultado.extend(der[j:])
    return resultado
```

Mergesort — visualización

Sobre [38, 27, 43, 3, 9, 82, 10]

◀ Dividir



Fusionar

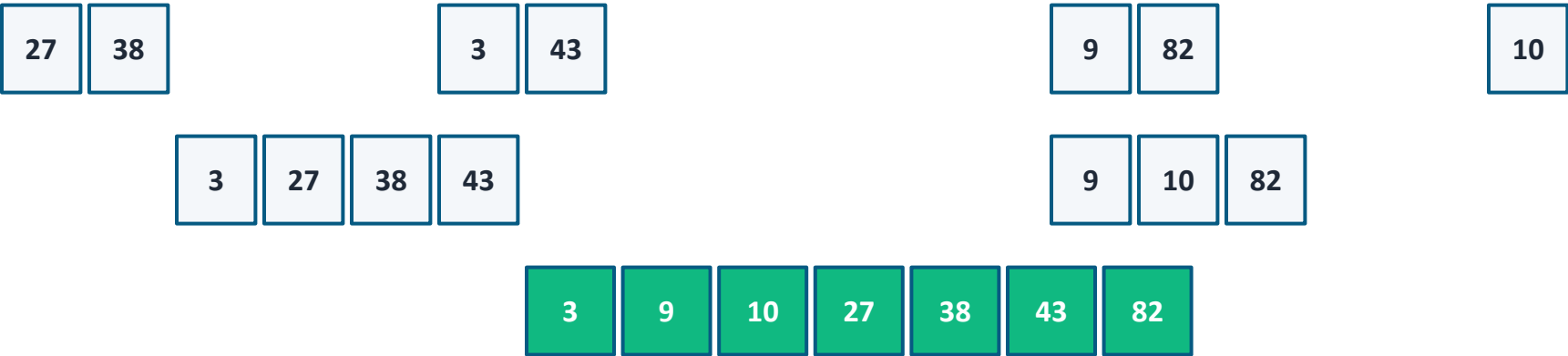


Tabla comparativa

Resumen de complejidades

Algoritmo	Mejor	Promedio	Peor	Memoria	Estable
Secuencial	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	—
Selección	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Inserción	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Burbuja	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sí

💡 Para listas pequeñas: inserción. Para listas grandes: quicksort o mergesort. Cuando se requiere estabilidad: mergesort.

PARTE 3.3

Aplicaciones prácticas

Python, NumPy y pandas en acción

🕒 Tiempo estimado: 15 minutos

Python nativo: sorted() y bisect

Implementaciones eficientes ya disponibles

Timsort

Python usa internamente Timsort (híbrido de mergesort e inserción). $O(n \log n)$ en peor caso, $O(n)$ si los datos están casi ordenados.

```
●●●
# ORDENAR – Timsort ( $O(n \log n)$ )
numeros = [5, 2, 8, 1, 9, 3]
print(sorted(numeros))           # [1, 2, 3, 5, 8, 9]  nueva lista
numeros.sort()                  # in-place
print(sorted(numeros, reverse=True)) # [9, 8, 5, 3, 2, 1]

# Ordenar por clave personalizada
alumnos = [{"nombre": "Ana", "calif": 8.5},
            {"nombre": "Luis", "calif": 9.2}]
print(sorted(alumnos, key=lambda a: a["calif"], reverse=True))

# BUSCAR – bisect (búsqueda binaria  $O(\log n)$ )
import bisect
ordenados = [1, 3, 5, 7, 9, 11]
pos = bisect.bisect_left(ordenados, 7) # 3
bisect.insort(ordenados, 8)            # inserta manteniendo orden
```

NumPy: cómputo numérico a gran escala

Vectorizado en C, hasta 10× más rápido

```
import numpy as np

datos = np.array([23, 5, 17, 8, 42, 11, 30])

# Ordenamiento (quicksort por defecto)
ordenados = np.sort(datos)           # [ 5  8 11 17 23 30 42]
print(np.sort(datos, kind="mergesort")) # otro algoritmo

# Obtener índices que ordenarían el arreglo
indices = np.argsort(datos)          # [1 3 5 2 0 6 4]

# Búsqueda binaria vectorizada
pos = np.searchsorted(ordenados, 17)  # 3

# Buscar varios valores en una sola llamada
buscar = np.array([8, 30, 100])
print(np.searchsorted(ordenados, buscar)) # [1 5 7]
```

¿Cuándo usar NumPy?

- ✓ Datos numéricos
- ✓ Grandes volúmenes
- ✓ Operaciones vectorizadas
- ✓ Cálculos científicos

Algoritmos disponibles

```
kind="quicksort"
      (default)
kind="mergesort"
kind="heapsort"
kind="stable"
```

pandas: análisis de datos tabulares

DataFrames con índices etiquetados

```
import pandas as pd

alumnos = pd.DataFrame({
    "matricula": [101, 102, 103, 104, 105],
    "nombre":    ["Ana", "Luis", "Eva", "Diego", "Sofía"],
    "calif":     [8.5, 9.2, 7.8, 6.4, 9.7],
    "carrera":   ["ICO", "ICO", "ISC", "ICO", "ISC"],
})

# Ordenar por calificación (descendente)
top = alumnos.sort_values(by="calif", ascending=False)

# Ordenar por múltiples criterios
combinado = alumnos.sort_values(
    by=["carrera", "calif"], ascending=[True, False])

# Búsqueda con filtros booleanos
aprobados = alumnos[alumnos["calif"] >= 9]
sofia = alumnos[alumnos["nombre"] == "Sofía"]
```

Lectura de archivos

```
pd.read_csv(...)
pd.read_excel(...)
pd.read_json(...)
pd.read_sql(...)
```

Operaciones útiles

```
sort_values()
sort_index()
filtros booleanos
groupby() + agg()
```

Caso integrador

Análisis de calificaciones — búsqueda + ordenamiento



```
import random, bisect

# 1. Generar 30 calificaciones aleatorias entre 5.0 y 10.0
random.seed(42)
calificaciones = [round(random.uniform(5.0, 10.0), 1)
                  for _ in range(30)]

# 2. Ordenar usando Timsort (O(n log n))
calificaciones_ord = sorted(calificaciones)

# 3. Calcular estadísticas usando la lista ordenada
minimo = calificaciones_ord[0]
maximo = calificaciones_ord[-1]
mediana = calificaciones_ord[len(calificaciones_ord) // 2]
promedio = sum(calificaciones_ord) / len(calificaciones_ord)

# 4. ¿Cuántos aprobaron? – búsqueda binaria
indice_6 = bisect.bisect_left(calificaciones_ord, 6.0)
aprobados = len(calificaciones_ord) - indice_6

# 5. Top 5
print(f"Top 5: {calificaciones_ord[-5:]}")
```

¿Qué se está haciendo?

1. Generar datos
2. Ordenar ($O(n \log n)$)
3. Calcular min, max, mediana y promedio
4. Contar aprobados con búsqueda binaria
5. Reportar el top 5

Ejercicios propuestos

Para practicar después de la clase

BÁSICO

- Búsqueda lineal que devuelva todos los índices
- Búsqueda binaria con posición de inserción
- Selección de cadenas sin distinguir mayúsculas
- Burbuja en orden descendente

INTERMEDIO

- Quicksort con mediana de tres como pivote
- Mergesort iterativo (sin recursión)
- Leer CSV con pandas, ordenar y guardar

AVANZADO

- Comparar los 6 algoritmos midiendo tiempos
- Raíz cuadrada entera por búsqueda binaria
- Investigar Timsort: ¿por qué es eficiente?

Para llevarse a casa

Ideas clave de la sesión



Búsqueda

Usa lineal si los datos NO están ordenados o son pocos. Usa binaria cuando los datos estén ordenados y se hagan muchas búsquedas.



Ordenamiento

Los algoritmos $O(n^2)$ son didácticos pero lentos. Para producción, prefiere quicksort, mergesort o el `sorted()` de Python.



Complejidad

Big-O describe cómo escala el algoritmo. $O(\log n)$ y $O(n \log n)$ son los grandes ganadores para datos grandes.



En la práctica

Python, NumPy y pandas ya implementan estos algoritmos de forma eficiente. Úsalos en proyectos reales, pero entiende cómo funcionan.

¿Preguntas?

Gracias por su atención